

DESARROLLO DE SOFTWARE

Clase IV: Fundamentos de la Programación

Variables, estructuras de control, funciones y paradigmas

2026



ALGORITMO, VARIABLES Y TIPOS DE DATOS

Un **algoritmo** es una secuencia lógica de pasos para resolver un problema.

Una **variable** es un espacio en memoria con un nombre y un valor.

Los **tipos de datos** definen qué clase de valor puede almacenar.

Enteros (int)

```
edad = 25
```

Decimales (float)

```
precio = 3.14
```

Texto (string)

```
nombre = "Ana"
```

Booleanos (bool)

```
activo = true
```

Listas / Arrays

```
nums = [1,2,3]
```

Objetos / Diccionarios

```
u = {id:1, rol:"admin"}
```

ESTRUCTURAS DE CONTROL

Las **estructuras de control** son mecanismos para controlar el flujo de ejecución del programa, como sentencias condicionales.

CONDICIONALES (if / else)

Ejecutan código solo si una condición es verdadera.

```
if (edad >= 18) {  
  print("Mayor de edad");  
} else {  
  print("Menor de edad");  
}
```

También: switch/case para múltiples condiciones.

BUCLES (loops)

Repiten un bloque de código mientras se cumple una condición.

```
// for: iteraciones fijas  
for (i=0; i<5; i++) {  
  print(i);  
}  
// while: condición variable  
while (activo) { ... }
```

También: forEach, do-while, iteradores.

FUNCIONES Y MODULARIZACIÓN

Una **función** es un bloque de código reutilizable que realiza una tarea específica.
La **modularización** divide el programa en partes pequeñas e independientes.

```
// Función con parámetros y retorno
function calcularIVA(precio, tasa) {
  const iva = precio * tasa;
  return precio + iva;
}

// Uso:
const total = calcularIVA(1000, 0.21);
// total = 1210
```

DRY — Don't Repeat Yourself

Si repites código en 3 lugares → creá una función.

SRP — Single Responsibility

Cada función hace UNA sola cosa.

KISS — Keep It Simple

Funciones cortas y fáciles de entender.

Tipos de funciones: puras, de orden superior, lambda/arrow, recursivas.

PARADIGMAS DE PROGRAMACIÓN

Imperativo / Procedural

Se indica CÓMO hacer algo paso a paso. El código es una secuencia de instrucciones. Ej: C, Pascal.

Orientado a Objetos (OOP)

Modela el mundo como objetos con atributos y comportamientos. Principios: encapsulamiento, herencia, polimorfismo. Ej: Java, Python, C#.

Funcional

Trata el cómputo como evaluación de funciones matemáticas. Sin efectos secundarios, inmutabilidad. Ej: Haskell, Elixir, partes de JS.

Declarativo

Se indica QUÉ se quiere lograr, no cómo. Ej: SQL ('dame todos los clientes con deuda > 0').

PARADIGMAS DE PROGRAMACIÓN

Existen diferentes maneras de organizar el código y estructurar los programas.

Los paradigmas de programación más comunes son:

se basa en la definición de funciones como bloques de código reutilizables.

**FUNCTIONAL
PROGRAMMING (FP)**

Haskell

03

Se basa en la creación de objetos que encapsulan datos y comportamiento.

**OBJECT-ORIENTED
PROGRAMMING**

Java, C#

02

**STRUCTURED
PROGRAMMING**

C, Pascal

01

Se basa en la descomposición de problemas en subproblemas más pequeños y manejables.

PROGRAMACIÓN ESTRUCTURADA

DISEÑO DESCENENTE (TOP- DOWN)



```
public class Main {  
  
    public static void main(String[] args) {  
        Scanner teclado = new Scanner(System.in);  
  
        int numero, suma = 0, cont = 0;  
        double promedio;  
  
        for (int i = 1; i <= 10; i++) {  
            System.out.println("Introduzca un numero: ");  
            numero = teclado.nextInt();  
  
            if (numero % 2 != 0) {  
                suma = suma + numero;  
                cont = cont + 1;  
            }  
  
            promedio = suma / cont;  
  
            System.out.println("La suma es: " + suma);  
            System.out.println("Los numeros impares fueron: " + cont);  
            System.out.println("El promedio es: " + promedio);  
        }  
    }  
}
```

ENTRADA

PROCESO

Sentencia Repetitiva (for/while).

Sentencia Selectiva (if/else)

RESULTADO

POO – PROGRAMACIÓN ORIENTADA A OBJETOS

DISEÑO DESCENENTE (TOP- DOWN)



INSTANCIA DE LA CLASE

```
package com.company;

public class App {

    public static void main(String[] args) {
        WashingMachine machineOne = new WashingMachine( brandName: "Laundatorium",
            modelName: "Washmeister", serialNumber: "GLS7774", capacity: 16 );

        System.out.println("La marca de la lavadora es: " + machineOne.getBrandName());
        System.out.println("El modelo de la lavadora es " + machineOne.getModelName());
        System.out.println("El numero de serie de la lavadora es: " + machineOne.getSerialNumber());
        System.out.println("La capacidad de la lavadora es de: " + machineOne.getCapacity());
    }
}
```

```
package com.company;
```

```
public class WashingMachine {
```

```
    private String brandName;
    private String modelName;
    private String serialNumber;
    private int capacity;
```

```
    public WashingMachine(String brandName, String modelName, String serialNumber,
        this.brandName = brandName;
        this.modelName = modelName;
        this.serialNumber = serialNumber;
        this.capacity = capacity;
    }
```

```
    public String getBrandName() {
        return brandName;
    }
```

```
    public void setBrandName(String brandName) {
        this.brandName = brandName;
    }
```

CLASES

ATRIBUTOS

CONSTRUCTORES

MÉTODOS

POO – PROGRAMACIÓN ORIENTADA A OBJETOS

POO se basa en la creación de objetos que encapsulan datos y comportamiento.

Las principales características de la POO son encapsulamiento, abstracción, herencia, polimorfismo, clases y objetos, y relaciones entre objetos.

Ventajas de utilizar POO

- **Reutilización de código:** Puedes crear y reutilizar clases y objetos en diferentes partes de tu aplicación, evitando la duplicación de código y mejorando la eficiencia.
- **Modularidad:** Al dividir tu aplicación en módulos o clases independientes, puedes mejorar la organización y facilitar el mantenimiento y la actualización del código.
- **Flexibilidad y escalabilidad:** La POO te permite extender y modificar fácilmente el comportamiento de tus objetos a medida que evolucionan los requisitos de tu aplicación.
- **Mantenibilidad:** La estructura y la organización de la POO hacen que el código sea



CARACTERÍSTICAS DE POO

Encapsulamiento

Agrupar datos (atributos) y el comportamiento asociado a ellos (métodos) dentro de una unidad llamada objeto.
Oculta la implementación interna de los objetos, exponiendo solo una interfaz pública (métodos) para interactuar con ellos.
Promueve la modularidad y la reutilización del código.

Abstracción

Representa las características esenciales de un objeto, ignorando detalles irrelevantes.
Permite crear modelos conceptuales más simples y fáciles de entender.
Facilita la creación de jerarquías de clases y la herencia.

Herencia

Permite a una clase heredar atributos y métodos de otra clase padre.
Promueve la reutilización del código y la especialización.
Facilita la creación de jerarquías de clases y la organización del código.

Polimorfismo

Permite que objetos de diferentes clases respondan al mismo mensaje de manera diferente.
Se implementa a través de:

- Sobrecarga de métodos: permite definir métodos con el mismo nombre pero diferentes parámetros o tipos de retorno.
- Sobrescritura de métodos: permite redefinir un método heredado de una clase padre en una clase hija.

POO – CLASES Y OBJETOS

Clases y objetos:

Las clases son los planos o plantillas a partir de los cuales se crean los objetos.

Los objetos son instancias individuales de una clase.

Las clases definen las características y el comportamiento de los objetos.

Los objetos encapsulan datos y métodos específicos.

Relaciones entre objetos:

Los objetos pueden relacionarse entre sí de diferentes maneras:

Asociación: un objeto utiliza o contiene otro objeto.

Agregación: un objeto es parte de otro objeto, pero mantiene su propia identidad.

Composición: un objeto es parte de otro objeto y no puede existir de forma independiente.



PROGRAMACIÓN FUNCIONAL

Expresamos nuestra lógica sin escribir controles de flujo (Ciclos o Condicionales)

Es funcional porque delega el control de flujos y condicionales a las funciones

```
1 package com.company;
2
3
4 import java.util.List;
5
6 public class Funcional {
7
8     public static void main(String[] args) {
9         List<Integer> numeros = List.of(78, 1, 4, 2, 67);
10        Long resultado = numeros.stream().filter(num -> num < 10).count();
11        System.out.println("Numeros menores a 10: " + resultado);
12    }
13 }
```

Conceptos

Funciones

PROGRAMACIÓN FUNCIONAL

Dividir la mayor cantidad posible de tareas en funciones, de esta forma estas tareas pueden ser usadas por otras funciones con diferentes objetivos.

El objetivo es conseguir lenguajes expresivos y matemáticamente elegantes, en los que no sea necesario bajar al nivel de la máquina para describir el proceso llevado a cabo por el programa, y evitar el concepto de estado del cómputo.

La secuencia de computaciones llevadas a cabo por el programa se rige única y exclusivamente por la reescritura de definiciones más amplias a otras cada vez más concretas y definidas.

Este paradigma, **por no contener datos mutables**, se caracteriza por ser usado para el manejo de información y no para la creación o modificación de la misma.

IA PARA PROGRAMADORES — GITHUB COPILOT Y MÁS

GitHub Copilot, lanzado en 2021, cambió para siempre el flujo de trabajo del desarrollador. En 2026, el 70% de los devs lo usa diariamente.

◆ Autocompletado inteligente

Completa líneas y bloques de código basándose en el contexto. Aprende de tu estilo de código.

◆ Generación desde comentarios

Escribís "// función que ordena array de objetos por precio" y Copilot genera el código completo.

◆ Explicación de código

Seleccionás código complejo o heredado y le pedís que lo explique en lenguaje natural.

◆ Generación de tests

Genera automáticamente casos de prueba unitarios para tus funciones con distintos escenarios.

CUESTIONARIO

1. ¿Qué es una variable y qué función cumplen los tipos de datos en la programación? Nombrar Ej. De tipos de datos
2. ¿Cuál es la diferencia fundamental entre una estructura de control condicional (if/else) y un bucle (loop)?
3. ¿Qué es una función y una modularización?
4. ¿Cuáles son los tipos de paradigmas que existen? Breve descripción de cada uno.

¡MUCHAS GRACIAS!

Clase IV: Fundamentos de la Programación

Próxima clase: consultar cronograma.

“Yo hago lo que usted no puede, y usted hace lo que yo no puedo. Juntos podemos hacer grandes cosas” –
Madre Teresa de Calcuta